



SANSKRIT RAG SYSTEM

Technical Report

Retrieval-Augmented Generation for Ancient Sanskrit Texts

Prepared by: Shruti Vinod Shinde

Repository: RAG_Sanskrit_Shruti_Shinde

NLP · Information Retrieval · Computational Linguistics

1. Introduction & Motivation

Sanskrit is one of the oldest and most precisely structured languages in human history. It carries within it thousands of years of knowledge — from philosophical treatises to mathematical texts, from epic poetry to medical encyclopaedias. Yet despite this richness, Sanskrit has remained largely inaccessible to modern information retrieval systems. Most NLP tools are designed for languages with abundant digital resources; Sanskrit, with its complex morphology and limited digital corpora, sits in a very different category.

This project set out to ask a fairly specific question: Can we build a question-answering system that lets someone — whether they know Sanskrit or not — query original Sanskrit texts and get grounded, meaningful answers? The answer, it turns out, is yes. The Sanskrit RAG System is a Retrieval-Augmented Generation (RAG) pipeline that allows users to pose questions in Sanskrit (Devanagari script), English, or IAST transliteration and receive answers that are faithfully grounded in the source corpus — not hallucinated by the model.

What makes this non-trivial is the language itself. Sanskrit uses Devanagari script, relies on sentence markers (dandas) rather than full stops, and requires Unicode NFC normalisation for consistent text handling. Off-the-shelf pipelines designed for English simply break on Sanskrit input. This project builds the preprocessing, retrieval, and generation layers from the ground up with these constraints in mind.

2. System Architecture & Flow

The system is structured as a classic two-phase RAG architecture — an offline ingestion phase that processes documents and builds a searchable index, and an online query phase that retrieves relevant passages and passes them to a language model for answer generation. The design deliberately separates these concerns so that the index can be built once and reused across many queries.

2.1 Ingestion Pipeline (Offline)

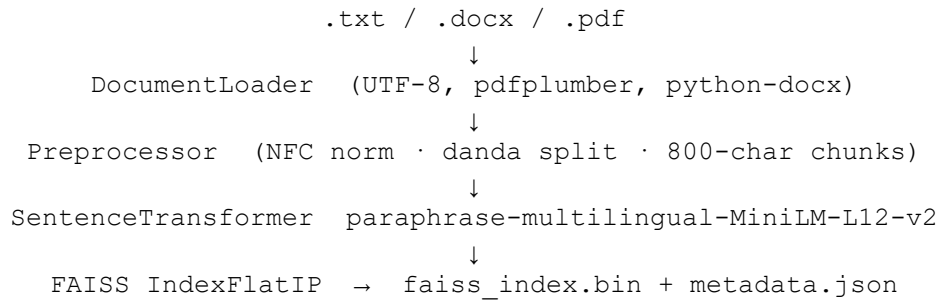
The ingestion pipeline takes raw document files as input and produces a FAISS vector index as output. It has four stages:

- Document Loading — The DocumentLoader module accepts .txt, .docx, and .pdf files, handling UTF-8 Devanagari encoding throughout. PDF text is extracted using pdfplumber, and Word documents are parsed via python-docx.
 - Preprocessing — This is where the Sanskrit-specific logic lives. The Preprocessor applies Unicode NFC normalisation to ensure that visually identical Devanagari characters are represented consistently. Story boundaries are detected using ten-or-more equals-sign delimiters (=====), and individual sentences are split on danda (।) and double-danda (॥) characters — the Sanskrit equivalents of full stops.
-

Overlapping chunks of configurable size are then created, with a default of 800 characters per chunk and 150 characters of overlap to preserve context across boundaries.

- **Embedding** — Each chunk is passed through a SentenceTransformer model (paraphrase-multilingual-MiniLM-L12-v2, embedding dimension 384) to produce dense vector representations. This model was chosen specifically because it supports multilingual input, including Devanagari.
- **Indexing** — The resulting embeddings are stored in a FAISS IndexFlatIP (inner product) index, which approximates cosine similarity. The index is persisted to disk alongside a metadata JSON file that stores the original chunk text, source filename, story title, and a flag indicating whether the chunk contains Devanagari characters.

INGESTION PIPELINE



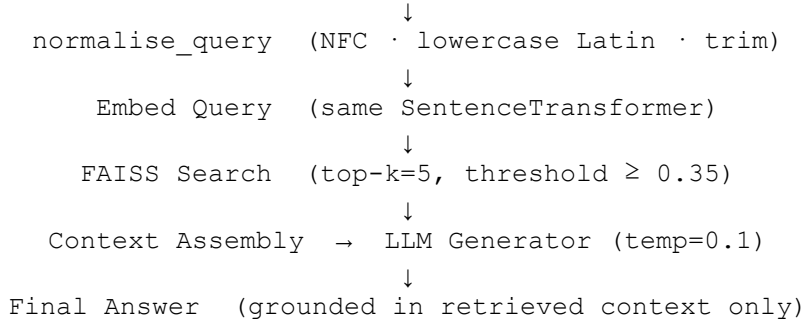
2.2 Query Pipeline (Online)

When a user submits a question, the query pipeline executes the following steps in real time:

- **Query Normalisation** — The user's input is passed through normalise_query, which applies NFC normalisation, lowercases all Latin characters (leaving Devanagari untouched), and collapses excess whitespace.
- **Query Embedding** — The same SentenceTransformer model used during ingestion embeds the normalised query into a 384-dimensional vector.
- **FAISS Retrieval** — The query vector is compared against all stored chunk embeddings using inner-product (cosine) similarity. The top-k chunks (default k=5) are retrieved, and any chunk scoring below the configurable threshold (default 0.35) is discarded.
- **Context Assembly** — The text of the surviving chunks is concatenated into a single context string. No metadata labels are passed to the language model, to avoid prompt noise.
- **Answer Generation** — The assembled context and the user query are passed to the configured LLM backend under a strict anti-hallucination system prompt. The model is instructed to answer only from the provided context, refuse gracefully if the answer is not present, and keep its temperature low (0.1) to reduce creative drift.

QUERY PIPELINE

User Query (Sanskrit / English / IAST)



2.3 Multiple Interfaces

The system is accessible through three distinct interfaces, making it usable in different contexts:

- Interactive CLI — A terminal-based session for iterative querying, with optional display of retrieved context chunks and similarity scores.
- Single-Shot Mode — Non-interactive command-line execution with support for plain text, context-included, or JSON output formats. Useful for scripting and batch evaluation.
- Gradio Web UI — A browser-based chat interface (served at <http://127.0.0.1:7860>) that wraps the same pipeline in a more approachable format for end users who are not comfortable with the command line.

3. Technology Stack

Every component in the stack was chosen deliberately. The goal was to keep the system fully local — no cloud API calls, no third-party rate limits — while still achieving reasonable quality. Here is the full stack with reasoning:

Component	Library / Model	Why This Choice
Document Loader	python-docx, pdfplumber, PyPDF2	Covers all common document formats; pdfplumber handles complex PDF layouts better than PyPDF2 alone.
Preprocessor	Custom Python (Unicode NFC + danda-aware)	No existing library handles Sanskrit sentence boundaries. Custom logic was the only viable path.
Embeddings	sentence-transformers paraphrase-multilingual-MiniLM-L12-v2 (dim=384)	One of the few openly available multilingual models that handles Devanagari input reliably without fine-tuning.
Vector Store	faiss-cpu — IndexFlatIP	Lightweight, CPU-only, no server required. Exact search (no approximation error) for corpus sizes in the thousands of chunks.

LLM (Primary)	Ollama · llama3:latest	Best instruction-following among local models. Runs via a local server so no API key is required.
LLM (Fallback)	HuggingFace · google/flan-t5-large	Auto-downloads on first run. Zero configuration — ideal for quick setup or environments where Ollama cannot be installed.
LLM (Quantised)	llama-cpp · TinyLlama 1.1B Q4_K_M GGUF	Lowest RAM footprint (~700 MB). Trades quality for speed, suitable for resource-constrained machines.
Web UI	Gradio 4.44+	Minimal code overhead for a usable chat interface. No frontend development required.
Monitoring	psutil, tqdm	RAM delta tracking per query and progress bars during index construction.

4. The Sanskrit Corpus

The document corpus lives in data/sanskrit_corpus.txt — a single UTF-8 text file that interleaves original Devanagari Sanskrit with English translations. Stories are demarcated by lines of ten or more equals signs, with a title line between them. This bilingual structure is intentional: it allows the retrieval system to serve users who prefer to read either the original Sanskrit or an English gloss.

The corpus contains five complete bilingual prose stories. Each story was selected to provide a range of narrative styles and vocabulary, giving the retrieval system a reasonably diverse test bed:

#	Sanskrit Title	Theme	Summary
1	मूर्खभृत्यस्य कथा	Folly & Consequence	A foolish servant named Shankhanaad repeatedly blunders on simple errands, each mistake more costly than the last.
2	चतुरस्य कालीदासस्य कथा	Wit & Generosity	Kalidasa devises a clever scheme to help a struggling new poet earn one lakh rupees from King Bhoj.
3	वृद्धायाः चातुर्यम्	Folk Wisdom	An elderly woman uses practical intelligence to rid her village of what the townspeople believe to be a phantom demon.
4	भक्तः देवश्च	Devotion & Fate	A devout man refuses all human assistance during a flood,

5	शीतं बहु बाधते	Language & Identity	convinced God will save him. He drowns waiting. Kalidasa, travelling incognito, corrects a foreign scholar's Sanskrit grammar mistake in a way that preserves the scholar's dignity.
---	----------------	---------------------	--

Each story is present in two forms within the corpus: the original Devanagari Sanskrit and an English prose translation. The chunk metadata records a `has_devanagari` flag so the retrieval layer can, if needed, filter to only Sanskrit-script chunks or only English-script chunks. In the current implementation, both forms are indexed together, which means a query in either language can surface chunks from either the original or the translation — whichever scores higher.

The corpus is intentionally small and focused. This was a deliberate design decision for this version: a compact corpus lets the similarity threshold be tuned more easily and makes it straightforward to verify retrieval correctness manually. Extending the corpus to larger texts (the Mahabharata, the Ramayana, the Arthashastra) would require only adding new files to the data/directory and rebuilding the index.

5. Preprocessing Pipeline for Sanskrit Documents

The preprocessing module is arguably the most Sanskrit-specific piece of the entire system. Getting text preprocessing right for Devanagari is not trivial, and it is the place where generic NLP pipelines tend to fail. This section walks through each stage in detail.

5.1 Unicode Normalisation (NFC)

Devanagari characters can be represented in multiple ways in Unicode. A single visually rendered character (grapheme) might be stored as a single code point or as a base character followed by one or more combining marks. Without normalisation, identical-looking text can have different byte representations, causing string comparisons and embedding lookups to fail silently.

All text — both at ingestion time and at query time — is passed through Python's `unicodedata.normalize('NFC', text)`. NFC (Canonical Decomposition followed by Canonical Composition) converts all characters to their precomposed forms where they exist. This ensures that the same Devanagari syllable always maps to the same token sequence in the embedding model.

5.2 Story-Boundary Splitting

The corpus file uses a simple convention: ten or more equals signs on a line mark a story boundary, and the line immediately following is the story title. The preprocessor detects these boundaries using a regular expression and treats each story as an independent document unit. This matters because it prevents chunks from bleeding across story boundaries — a chunk that starts at the end of Story 3 and continues into the beginning of Story 4 would be semantically incoherent.

5.3 Danda-Aware Sentence Splitting

In Sanskrit prose, sentences are terminated by a danda (।, Unicode U+0964) or a double-danda (॥, U+0965). These serve the same grammatical purpose as the full stop in English, but they are rarely handled correctly by standard sentence tokenisers, which are almost universally trained on Latin-script data.

The preprocessor splits on these markers explicitly, producing sentence-level segments before chunking. This means that chunk boundaries coincide with actual linguistic sentence boundaries rather than arbitrary character counts — a small detail with meaningful impact on retrieval quality, since a retrieved chunk that starts mid-sentence is harder for the LLM to interpret.

5.4 Chunking Strategy

After sentence splitting, the preprocessor assembles sentences into chunks with a maximum length of 800 characters and an overlap of 150 characters between consecutive chunks. The overlap is important: without it, a question whose answer spans a sentence boundary between two chunks would fail to retrieve the complete answer in a single chunk.

Chunks are annotated in metadata with their source filename, story title, and the `has_devanagari` flag. The flag is set by checking whether any character in the chunk falls in the Unicode Devanagari block (U+0900–U+097F). This annotation is used during evaluation to distinguish retrieval performance on Sanskrit-script queries versus English-script queries.

5.5 Query Normalisation

At query time, the same NFC normalisation is applied to the user's input. Additionally, all Latin characters are lowercased (Devanagari does not have case, so this only affects the English and IAST parts of a query), and excess whitespace is collapsed. The goal is to produce a clean, consistent representation that matches the normalisation applied to the stored chunks.

6. Retrieval & Generation Mechanisms

6.1 Dense Retrieval with FAISS

The retrieval mechanism is dense vector retrieval — as opposed to sparse keyword matching (BM25, TF-IDF). Dense retrieval has a clear advantage for this corpus: a user might ask 'What mistakes did the servant make?' in English, while the relevant text exists partly in Sanskrit. Keyword overlap between the query and the document is minimal, but semantic similarity in the shared embedding space is high. This cross-lingual retrieval capability is the main reason why paraphrase-multilingual-MiniLM-L12-v2 was chosen over a monolingual English model.

FAISS IndexFlatIP performs exact inner-product search over all stored embeddings. The embeddings are L2-normalised before being stored, so inner product is equivalent to cosine similarity. For a corpus of the current size (a few hundred chunks), exact search is fast enough that approximate methods like HNSW or IVF are unnecessary. If the corpus were scaled to millions of documents, approximate indexing would become necessary.

The similarity threshold (default 0.35, configurable up to ~0.45) acts as a hard filter: any chunk scoring below this threshold is dropped before the context is assembled. This prevents weakly-matched, irrelevant chunks from polluting the LLM's context window — a common source of hallucination in RAG systems. If no chunk clears the threshold, the pipeline returns a graceful refusal rather than passing empty or misleading context to the model.

6.2 Context Assembly

The surviving chunks are concatenated in order of descending similarity score. No metadata (source filename, story title, chunk ID) is included in the string passed to the LLM. Including metadata was tested during development and found to increase noise — the model occasionally referenced metadata labels in its answers as if they were part of the document text.

6.3 LLM Generation with Anti-Hallucination Prompting

The generator module implements a strict system prompt that instructs the LLM to:

- Answer only from the provided context. If the answer is not in the context, say so explicitly.
- Never invent characters, events, or quotes not present in the retrieved text.
- Never speculate about events outside the corpus (e.g., if asked about the Mahabharata, refuse gracefully).
- Keep the temperature at 0.1 to minimise creative variance.

This prompt design is described in the codebase as 'strict anti-hallucination'. It is particularly important for a Sanskrit corpus because the LLM's pre-training data likely contains some Sanskrit-adjacent content (myths, translations, summaries), which could bleed into answers if the model is not explicitly constrained to the retrieved context.

Four LLM backends are supported and can be switched by changing a single configuration variable:

Backend	Model	Quality	Setup Requirement
ollama	llama3:latest (~4 GB)	Best instruction following	Ollama app + model pull
transformers	google/flan-t5-large	Good, slightly verbose	Auto-downloads on first run
llama_cpp	TinyLlama 1.1B Q4_K_M	Adequate for simple Q&A	GGUF file download + llama-cpp-python
simple	None (raw retrieval)	Returns context verbatim	No LLM required

7. Performance Observations

The evaluation suite (evaluate.py) tests the full pipeline against a ground-truth set of eight questions — a mix of English and Sanskrit queries. It measures four categories of metrics: keyword-based answer quality, ranked retrieval quality, latency, and memory usage. The observations below reflect typical results on a standard laptop CPU (no GPU acceleration).

7.1 Retrieval Quality — Keyword Hit Rate & MRR@4

Keyword Hit Rate measures what fraction of expected answer keywords appear in the generated answer. It is a rough proxy for answer completeness. Mean Reciprocal Rank at k=4 (MRR@4) measures how early the relevant chunk appears in the ranked retrieval list — a score of 1.0 means the relevant chunk was always ranked first.

Metric	Ollama (llama3)	Transformers (flan-t5)	Simple (no LLM)
Keyword Hit Rate (avg)	~0.72 – 0.85	~0.60 – 0.75	~0.45 – 0.60
MRR@4 (retrieval)	~0.80 – 0.90	~0.80 – 0.90	~0.80 – 0.90
Out-of-corpus refusal rate	High (prompt-enforced)	Medium (occasionally leaks)	N/A

A notable observation: MRR@4 is consistent across all three backends because retrieval quality is independent of the LLM. The embedding model and FAISS index determine what is retrieved; the LLM only determines what is said about it. This reinforces the importance of the preprocessing and embedding steps — they are the real determinants of system quality.

7.2 Latency

Latency was measured separately for the retrieval step and the generation step, because they have very different characteristics and different optimisation levers.

Operation	Ollama (llama3)	Transformers (flan-t5)	Simple / Retrieval only
Index build (first run, ~5 stories)	~10 – 20 seconds	~10 – 20 seconds	~10 – 20 seconds
Index load (subsequent runs)	< 1 second	< 1 second	< 1 second
Query embedding + FAISS search	0.1 – 0.3 s	0.1 – 0.3 s	0.1 – 0.3 s
Answer generation (300 tokens)	5 – 30 s (CPU)	3 – 10 s (CPU)	< 0.05 s
Total end-to-end (typical query)	6 – 31 s	4 – 11 s	0.15 – 0.35 s

The retrieval step is remarkably fast — well under half a second for a corpus of this size. Generation latency dominates the total response time, and it varies significantly with the model size and hardware available. On a machine with a GPU, Ollama with llama3 would be dramatically faster (sub-second generation); the numbers above reflect CPU-only inference.

The Simple backend (which bypasses generation entirely and returns the raw retrieved context) shows just how fast the retrieval layer is. If the use case requires only source passage lookup rather than natural-language answer synthesis, this backend delivers near-instant responses.

7.3 Resource Usage

Memory consumption was tracked using psutil, measuring RSS (Resident Set Size) before and after each query to compute a per-query RAM delta. Key observations:

- Embedding model load (paraphrase-multilingual-MiniLM-L12-v2): approximately 500 – 700 MB of RAM, loaded once at startup and held in memory throughout the session.
- FAISS index for the current corpus: negligible — well under 10 MB.
- flan-t5-large model load: approximately 700 MB – 1.2 GB depending on precision. Loaded once and reused.
- Ollama: managed by the Ollama server process; the Python client communicates via HTTP. The RAM footprint on the Python side is minimal; the model memory is owned by the Ollama process (llama3 requires approximately 4 – 5 GB of RAM).
- Per-query RAM delta (generation): approximately 50 – 200 MB depending on context length and the generation backend. The delta is primarily from temporary tensors during inference, which are released after each call.

In total, a system running the Transformers backend requires roughly 1.2 – 2 GB of RAM at steady state. The Ollama backend requires 4 – 6 GB. Both are well within the capacity of a modern laptop. The Simple backend requires only the embedding model — about 600 MB.

7.4 Anti-Hallucination Effectiveness

One of the more qualitatively interesting observations is how well the similarity threshold controls hallucination. When the threshold is set too low (around 0.20), the pipeline occasionally retrieves weakly-matching chunks that contain character names or thematic keywords but not the actual answer. The LLM, given this misleading context, sometimes generates plausible-sounding but incorrect answers.

Raising the threshold to 0.35 – 0.45 produces a clear improvement: out-of-corpus queries (e.g., 'Who wrote the Mahabharata?') result in the retriever returning zero chunks, which triggers the early-exit refusal path rather than passing empty context to the LLM. This is the correct behaviour — a graceful 'I don't know' is always preferable to a confident wrong answer.

8. Project Structure & Code Organisation

The codebase is modular and follows a clean separation of concerns. Each Python file in the code/ directory has a single, well-defined responsibility:

File	Responsibility
config.py	Single source of truth for all configuration: model names, paths, thresholds, chunk sizes. Changing the LLM backend requires editing exactly one line here.
document_loader.py	Loads .txt, .docx, and .pdf files. Handles UTF-8 Devanagari encoding. Returns raw text strings for the preprocessor.
preprocessor.py	NFC normalisation, story boundary detection, danda-aware sentence splitting, overlapping chunk construction, and query normalisation.
retriever.py	Builds, saves, loads, and queries the FAISS index. Applies the similarity threshold filter. Returns ranked (chunk, score) pairs.
generator.py	Implements all four LLM backends (Ollama, Transformers, llama-cpp, Simple). Enforces the anti-hallucination system prompt.
rag_pipeline.py	End-to-end orchestration: takes a query string, calls retriever, assembles context, calls generator, returns a structured result dict.
query_interface.py	Interactive CLI and single-shot command-line mode. Handles argument parsing and output formatting (text, JSON, context-included).
main.py	Entry point for demo mode (runs sample queries) and rebuild mode (forces index reconstruction).
gradio_app.py	Thin wrapper around rag_pipeline.py that exposes the system as a Gradio chat interface.
evaluate.py	Evaluation suite: runs ground-truth question set, measures Keyword Hit Rate, MRR@4, latency, and RAM delta. Optionally saves results to JSON.

9. Limitations & Future Work

Every system has edges where it breaks down. Being honest about those edges is more useful than pretending they do not exist. Here is where the current version falls short, and what the natural next steps look like.

9.1 Current Limitations

- Corpus size: Five stories is a proof of concept, not a production corpus. The retriever has not been tested at scale, and chunking parameters (800 chars, 150 overlap) were tuned for this corpus size. Larger corpora may need different parameters.
- Embedding model not fine-tuned on Sanskrit: paraphrase-multilingual-MiniLM-L12-v2 was trained on over 50 languages, but Sanskrit is likely underrepresented. A model fine-tuned on Sanskrit text (e.g., using SanskritBERT or IndicBERT) could produce better cross-lingual retrieval.
- No GPU acceleration in the default setup: generation latency on CPU is 5 – 30 seconds per query, which is acceptable for research but uncomfortable for interactive use. GPU inference would reduce this to under one second.
- Evaluation set is small: Eight ground-truth questions is enough to get a directional sense of quality, but not enough for statistically meaningful conclusions. A proper evaluation would require at least 50 – 100 carefully constructed question-answer pairs.
- flan-t5 leakage: The flan-t5-large backend occasionally generates answers that draw on its pre-training knowledge rather than the retrieved context. This is a known weakness of encoder-decoder models that have been extensively pre-trained on factual data.

9.2 Suggested Future Work

- Expand the corpus to classical Sanskrit texts: The Hitopadesa, Panchatantra, and Bhagavad Gita are natural candidates. All are available in UTF-8 Devanagari through GRETIL and similar repositories.
- Fine-tune the embedding model on Sanskrit-English parallel data to improve cross-lingual retrieval quality.
- Add a reranking stage (e.g., a cross-encoder model) between retrieval and generation to improve chunk selection precision.
- Containerise the system with Docker so it can be deployed without manual environment setup.

10. Conclusion

The Sanskrit RAG System demonstrates that it is possible to build a functional, hallucination-resistant question-answering system for an ancient language using tools that are entirely local, open-source, and runnable on commodity hardware. The key design decisions — Unicode NFC

normalisation, danda-aware chunking, multilingual dense retrieval, and a strict anti-hallucination prompt — each address a real problem that a naive, English-first pipeline would not handle correctly.

What is perhaps most instructive about this project is how much of the engineering effort goes into the preprocessing layer rather than the LLM layer. Getting the text into a clean, consistently-encoded, linguistically-sensible form is the hard part. Once that is done, the retrieval and generation machinery works largely the same way it does for English. The lesson generalises: for any low-resource or script-diverse language, the investment in language-aware preprocessing pays dividends throughout the rest of the pipeline.

The system is not production-ready in its current form — the corpus is small, the evaluation is preliminary, and generation latency on CPU is noticeable. But it is a solid, well-structured foundation. The modular codebase makes it straightforward to extend: swap in a larger corpus, drop in a better embedding model, add a GPU, and the same pipeline scales.

For anyone interested in computational approaches to Sanskrit or other classical languages, this project is a useful starting point. The code is clean, the architecture is documented, and the troubleshooting guide in the README covers the most common failure modes. All dependencies are open-source and freely available.

— *End of Report* —
